

EC 8207: Applied Big Data Engineering Mini Project Report

Smart City Traffic & Congestion Monitoring System

EG/2020/4149 – Randula R.D.

GitHub Link: <https://github.com/Dhanith-Randula/Smart-City-Traffic.git>

Video Link: <https://drive.google.com/file/d/1XKYjwkOcecJjZKlM8E0cQJaH0Qa0DA4J/view?usp=sharing>

1 INTRODUCTION

Urban traffic congestion is growing challenge in modern cities such as Colombo. Increasing vehicle usage leads to delays, fuel wastage, and environmental pollution. Traditional traffic monitoring systems are often reactive and lack real time intelligence, making it difficult for authorities to respond effectively.

This project presents Smart City Traffic & Congestion Monitoring System that uses modern Big Data pipeline to simulate, process, and analyze real time traffic data. System integrates real time stream processing with batch analytics to provide both immediate insights and historical reporting. Architecture follows a Lambda style data pipeline, where data is continuously ingested, processed in real time, stored for persistence, and later analyzed through scheduled batch jobs. System helps identify congestion patterns, generate alerts, and provide actionable recommendations for traffic management.

2 SYSTEM ARCHITECTURE OVERVIEW

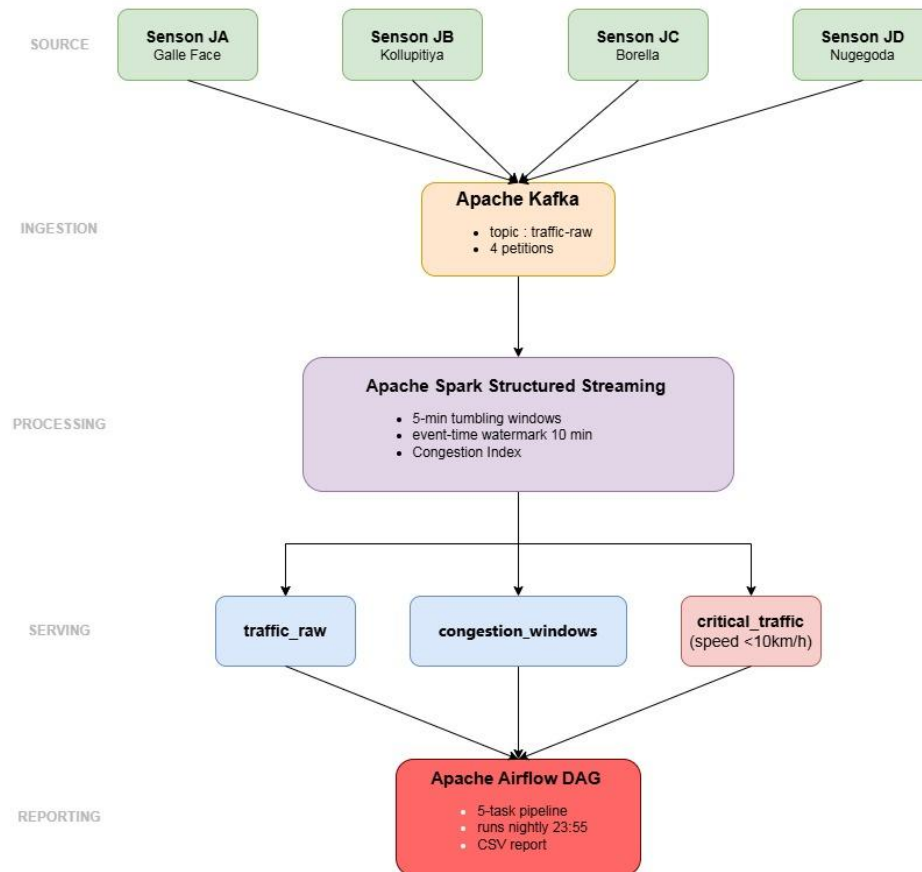


Figure 1: Architecture flow diagram

The system is composed of multiple interconnected layers:

- Source Layer: Simulated traffic sensors generating real-time data
- Ingestion Layer: Apache Kafka for streaming data ingestion
- Processing Layer: Apache Spark Structured Streaming
- Storage Layer: PostgreSQL database
- Orchestration Layer: Apache Airflow
- Reporting Layer: CSV-based analytical reports

Traffic data flows from sensors into Kafka topics, where it is consumed by Spark for real time processing. Processed results are stored in PostgreSQL and later analyzed by Airflow to generate daily reports.

3 JUSTIFICATION OF TECHNOLOGY STACK

Selected technologies are widely used in industry and were chosen based on scalability, reliability, and suitability for real time data processing.

3.1 Apache Kafka (Data Ingestion)

Apache Kafka is used as ingestion layer to handle high throughput real time data streams.

Reasons for selection:

- Supports real time data streaming
- High scalability through partitioning
- Fault tolerant and distributed architecture
- Decouples producers and consumers

Kafka efficiently handles continuous data from multiple sensors and ensures reliable delivery to downstream systems.

3.2 Apache Spark Structured Streaming (Processing Layer)

Apache Spark Structured Streaming is used real time data processing.

Reasons for selection:

- Supports stream processing with SQL like operations
- Provides window based aggregations
- Handles large scale data efficiently
- Built in support for event time processing and watermarking

Spark processes incoming data in near real time, calculates congestion metrics, detects anomalies such as low speed conditions.

3.2 PostgreSQL (Storage Layer)

PostgreSQL used as the storage system for structured data.

Tables used:

- **traffic_raw:** Stores incoming sensor data
- **congestion_windows:** Stores aggregated window based results
- **critical_traffic:** Stores congestion alerts

Reasons for selection:

- Reliable relational database system
- Strong consistency and ACID compliance
- Easy querying for reporting and analytics
- Seamless integration with Spark and Airflow

3.3 Apache Airflow (Orchestration Layer)

Apache Airflow used to manage and schedule batch processing tasks.

Reasons for selection:

- Supports workflow scheduling using DAGs
- Enables automation of data pipelines
- Provides monitoring and logging features
- Easy integration with databases

Airflow executes nightly jobs to generate analytical reports from stored data.

3.4 Docker Compose (Deployment)

Docker Compose used to containerize and manage all services.

Benefits:

- Simplifies environment setup
- Ensures consistency across systems
- Allows easy scaling and debugging

4 REAL-TIME PROCESSING LOGIC

System processes streaming data using Spark Structured Streaming.

4.1 Window-Based Aggregation

A 5-minute tumbling window, applied to group data by time intervals. This enables calculation of average traffic metrics per junction.

4.2 Congestion Index Calculation

The congestion index is calculated using:

$$\text{Congestion Index} = \text{Average Vehicle Count} / \text{Average Speed}$$

Higher values indicate heavier congestion.

4.3 Alert Generation

Critical traffic condition is defined as:

- Average speed < 10 km/h

When this condition is met:

- Alert is generated
- Data is stored in critical_traffic table
- Optionally sent to Kafka alert topic

4.4 Data Flow Outputs

Spark produces three outputs:

- Raw traffic data → traffic_raw
- Aggregated window data → congestion_windows
- Alerts → critical_traffic

5 EVENT TIME VS PROCESSING TIME

Handling time correctly is essential in stream processing systems.

5.1 Event Time

Event time refers to time when the data was originally generated by the sensor. In this project, the timestamp included in data is used as the event time.

5.2 Processing Time

Processing time refers to time when the data is processed by the system (Spark).

5.3 Implementation in the System

System uses event time processing for accurate analysis. This ensures that data is grouped based on when it was generated, not when it was processed.

To handle late arriving data, Spark applies a:

- Watermark of 10 minutes

6 BATCH PROCESSING AND REPORTING

Apache Airflow ,used to orchestrate batch processing tasks.

6.1 Airflow DAG Workflow

The Airflow DAG performs the following steps:

- Extract data from PostgreSQL
- Aggregate traffic data by hour
- Identify peak traffic hours per junction
- Calculate total vehicle counts and alerts
- Generate a CSV report

6.2 Report Output

The system generates a CSV file containing:

- Peak traffic hour
- Average vehicle count
- Average speed
- Total daily vehicles
- Number of alerts

- Intervention recommendations

Example insight:

- Peak traffic occurs between 11:00 AM and 1:00 PM
- All junctions require intervention

6.3 Recommendation Logic

Based on congestion levels, the system generates recommendations such as:

- Deploy traffic police during peak hours
- Optimize traffic signal timing

7 ETHICS AND DATA PRIVACY CONSIDERATIONS

Smart city systems involve continuous monitoring, which raises important ethical concerns.

7.1 Privacy Risks

- Continuous traffic monitoring may lead to surveillance concerns
- Location data can reveal movement patterns
- Potential misuse of collected data

7.2 Data Governance Strategies

To address these concerns, following practices should be implemented:

❖ Data Anonymization

- Avoid storing personal or identifiable information
- Use only sensor-based aggregated data

❖ Security Measures

- Encrypt stored data
- Restrict database access

❖ Policy and Compliance

- Follow data protection regulations
- Define clear data usage policies

❖ Data Retention

- Store data only for required duration
- Delete outdated records

7.3 Ethical Responsibility

Organizations must ensure that:

- Data is used only for traffic management purposes
- Users are aware of data collection practices
- Systems are designed with transparency and accountability

8 CONCLUSION

This project successfully demonstrates the implementation of complete end to end Big Data pipeline for smart city traffic monitoring.

The system:

- Ingests real time traffic data
- Processes it using distributed stream processing
- Detects congestion and generates alerts
- Stores data for historical analysis
- Produces daily analytical reports

Integration of Kafka, Spark, PostgreSQL, and Airflow highlights importance of scalable data engineering solutions in real world applications. This project provides strong foundation in Big Data concepts and showcases how modern data pipelines can be used to solve real world urban challenges.